Apache Airflow

Exercise 1 — Employee Salary Processing DAG

employee_salary_processing

Name: Vaishali S
Program: Data Engineering | Batch: 2
Date: 2026-05-09

Topics Covered

DAG Trigger • Graph View • Overview & Run History • Terminal Output

1. Exercise Overview

About This Exercise

This exercise demonstrates creating and running an employee salary processing workflow DAG in Apache Airflow. The DAG reads employee salary data, calculates totals, identifies the highest-paid employee, and generates a salary report file — all orchestrated as a 5-step Python pipeline.

1.1 DAG Summary

Property	Value
DAG Name	employee_salary_processing
Schedule	Manual Trigger (no schedule set)
Latest Run	2026-05-09 09:26:37 — Success
Next Run	Not scheduled
Max Active Runs	0 of 16
Owner	airflow | Version: v1
Failed Tasks (24h)	0
Operator Used	PythonOperator (all 5 tasks)

1.2 Task Pipeline

The DAG runs 5 tasks in a linear sequence using PythonOperator:

- create_employee_file — Creates employees.txt with employee name and salary data
- read_employee_data — Reads and parses employee records from the file
- calculate_salary_expense — Computes total salary expenditure across all employees
- find_highest_salary — Identifies the employee with the highest salary
- generate_salary_report — Writes the final summary to salary_report.txt

```
create_employee_file >> read_employee_data >> calculate_salary_expense >> find_highest_salary
>> generate_salary_report
```

2. Screenshot 1 — Terminal: Docker Start & File Check

This terminal output shows starting the Airflow Docker container and then attempting to read /tmp/employees.txt from the host machine. The file is not found on the host, which is expected — the DAG writes files inside the container's filesystem.

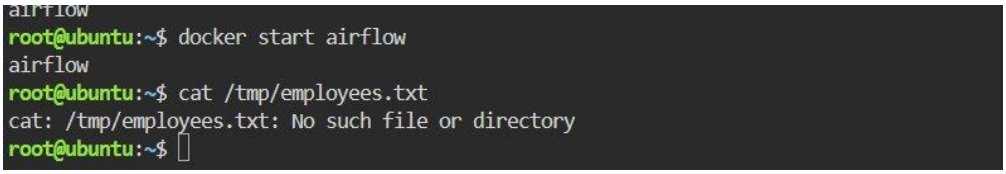

Figure 1: Terminal showing 'docker start airflow' and host-side cat of /tmp/employees.txt

Property	Value / Observation
Command	docker start airflow
Container Status	Started successfully
Host File Check	cat /tmp/employees.txt — No such file or directory
Reason	Files are written inside the Docker container, not on the host filesystem

3. Screenshot 2 — Terminal: Docker Exec & File Contents

This terminal output from inside the Docker container confirms the DAG correctly created both the input employees file and the output salary report in the /tmp directory.

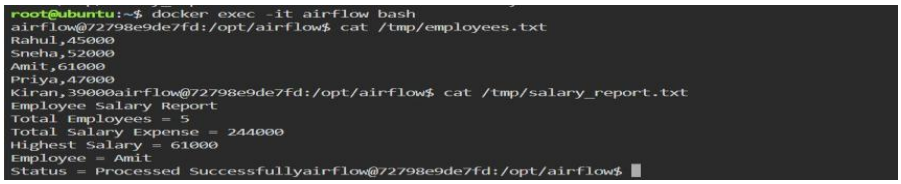

Figure 2: Docker exec terminal showing employees.txt and salary_report.txt contents

Property	Value / Observation
Command	docker exec -it airflow bash
Input File	/tmp/employees.txt
Employee Records	Rahul,45000 | Sneha,52000 | Amit,61000 | Priya,47000 | Kiran,39000
Output File	/tmp/salary_report.txt
Total Employees	5
Total Salary Expense	244000
Highest Salary	61000 — Employee: Amit
Status	Processed Successfully

3.1 Output Files Explained

employees.txt — Input File

Created by the create_employee_file task. Contains comma-separated name-salary pairs (e.g. Rahul,45000) used by all downstream tasks for processing.

salary_report.txt — Output File

Generated by generate_salary_report. Contains: Employee Salary Report / Total Employees = 5 / Total Salary Expense = 244000 / Highest Salary = 61000 / Employee = Amit / Status = Processed Successfully.

3.2 Terminal Commands

```
$ docker exec -it airflow bash
$ cat /tmp/employees.txt
Rahul,45000  |  Sneha,52000  |  Amit,61000  |  Priya,47000  |  Kiran,39000
$ cat /tmp/salary_report.txt
Total Employees = 5  |  Total Salary Expense = 244000  |  Highest Salary = 61000  |
Employee = Amit
Status = Processed Successfully
```

4. Screenshot 3 — DAG Graph View

The Graph View visualises the DAG's task dependency structure. Each box is a task node and arrows represent the execution order — left to right in this linear pipeline.

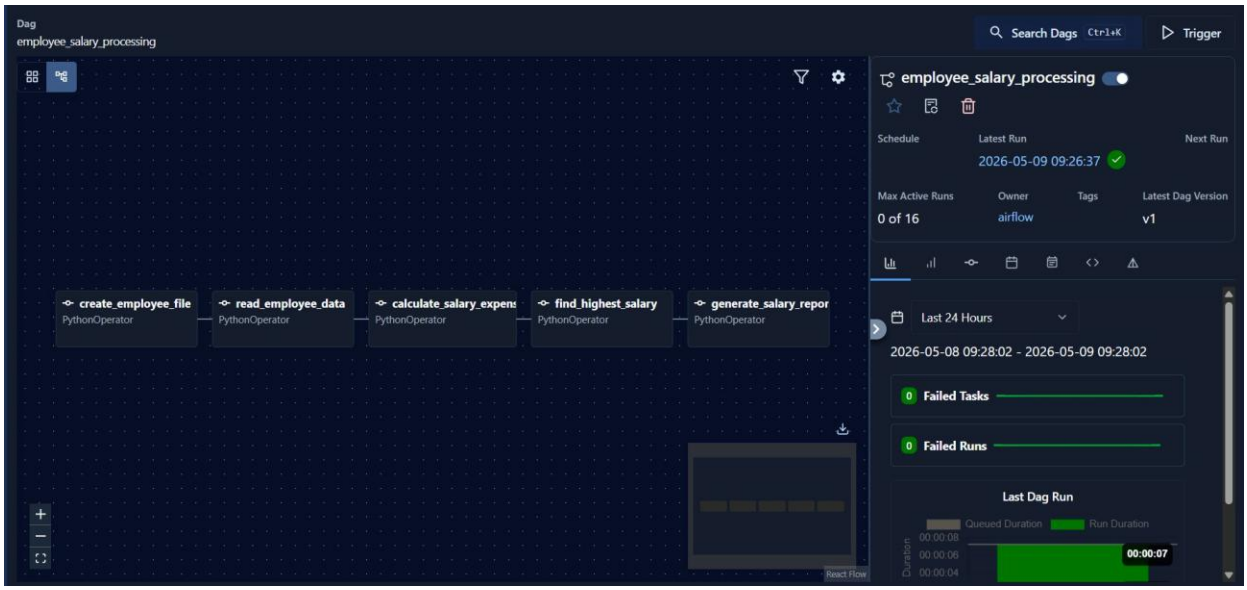

Figure 3: Graph View showing the 5-task linear pipeline for employee_salary_processing

Property	Value / Observation
View	Graph View (React Flow)
Task 1	create_employee_file — PythonOperator
Task 2	read_employee_data — PythonOperator
Task 3	calculate_salary_expense — PythonOperator
Task 4	find_highest_salary — PythonOperator
Task 5	generate_salary_report — PythonOperator
Dependency Type	Linear / sequential (left to right)

Failed Tasks	0 — all tasks completed successfully

5. Screenshot 4 — DAG Overview & Run History

The Overview tab shows task completion status, run durations, and failure statistics for the last 24 hours. All 5 tasks show green checkmarks indicating successful completion.

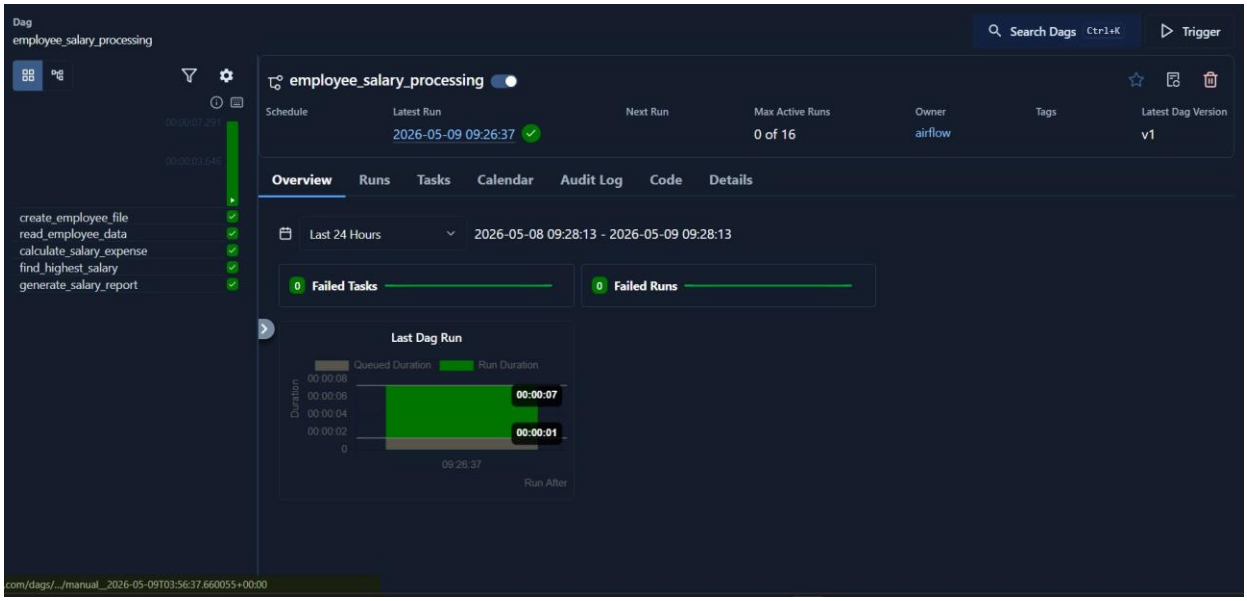

Figure 4: DAG Overview showing all 5 tasks with success checkmarks and run duration chart

Property	Value / Observation
Tasks	create_employee_file, read_employee_data, calculate_salary_expense, find_highest_salary, generate_salary_report
Task Status	All 5 — green checkmark (successful run)
Latest Run	2026-05-09 09:26:37 — Success
Run Duration	~7 seconds
Failed Tasks (24h)	0
Failed Runs (24h)	0
DAG Version	v1

5.1 Reading the Run Duration Chart

The bar chart shows the most recent DAG run with a green bar representing actual task execution time (~7 seconds). The near-zero grey queue duration means tasks were dispatched and picked up almost instantly by the executor, indicating no resource contention or scheduling delays.

6. Summary

This exercise successfully demonstrated a complete end-to-end data workflow in Apache Airflow. The employee_salary_processing DAG read employee salary records, computed totals, identified the highest-paid employee, and produced a verified output report — all with zero failures.

What Was Demonstrated	Outcome
Manual DAG Trigger	Triggered via UI — run completed successfully
5-task linear pipeline	All tasks completed with PythonOperator
File I/O operations	employees.txt created, read, report written
Total salary calculation	Rahul+Sneha+Amit+Priya+Kiran = 244000
Highest salary logic	Amit = 61000 confirmed in report
Zero failures	0 failed tasks, 0 failed runs in 24h